

TEXT MINING

L02. PREPROCESSING



SUZAN VERBERNE 2024

TODAY'S LECTURE

- Quiz about week 1
- Go from raw text to clean text
- Character encoding
- Regular expressions
- Tokenization and sentence splitting
- Lemmatization and stemming
- Edit distance (+ **exercise**)

QUIZ ABOUT WEEK 1

- If we use words as features in a text classification task, the resulting vectors are
 - a. Low-dimensional and dense
 - b. Low-dimensional and sparse
 - c. High-dimensional and dense
 - d. High-dimensional and sparse

QUIZ ABOUT WEEK 1

- If we use words as features in a text classification task, the resulting vectors are
- a. Low-dimensional and dense
 - b. Low-dimensional and sparse
 - c. High-dimensional and dense
 - d. **High-dimensional and sparse**

QUIZ ABOUT WEEK 1

- What does the long-tail distribution for text data refer to?
 - a. When we add a document to a collection, the number of unique terms will increase
 - b. In a given document, most of the terms will have a frequency of zero
 - c. In a given collection, there are many terms with a low frequency and few terms with a high frequency

QUIZ ABOUT WEEK 1

- What does the long-tail distribution for text data refer to?
 - a. When we add a document to a collection, the number of unique terms will increase
 - b. In a given document, most of the terms will have a frequency of zero
 - c. **In a given collection, there are many terms with a low frequency and few terms with a high frequency**

QUIZ ABOUT WEEK 1

- For which type of text processing task are capitalization and punctuation more useful?
 - a. For sequence labelling
 - b. For classification

QUIZ ABOUT WEEK 1

- For which type of text processing task are capitalization and punctuation more useful?
 - a. **For sequence labelling**
 - b. For classification

WHO FOUND A TEAM MATE?

➤ (please raise your hands)

FROM RAW TEXT TO CLEAN TEXT

GATHERING RAW TEXT

- Written text
 - Digitized (scanned) documents
(We need **OCR = optical character recognition**)
 - Born-digital documents
 - text, html, pdf, MS Word documents
- All text needs clean-up of some kind

DIGITAL INPUT NOT CLEAN

- Scanned text and born-digital PDFs might contain:
 - photos, tables, graphics
 - layout or design information
 - disclaimers, copyright statements
 - headers, footers
 - column and page breaks
 - **OCR errors**
 - **character encoding errors**
- Semi-structured text: text with **markup** (HTML, XML)

OPTICAL CHARACTER RECOGNITION (OCR)

TO THE TRVLY HO-
NORABLE AND RIGHT WOR-
THY KNIGHT SIR THOMAS SMITH,

TREASVRER for the Colonies and Com-
panies of VIRGINIA: and *Gouernour* of Mus-
couia, East-India, North-west Passage,
and SOMMER Ilands
Companies.



HONORABLE SIR, the wisest of Men, or rather the wisdom of God tells vs, that *there is a time for all things*: and that the great God, who at his owne will beganne Time it selfe, doth at his owne time beginne all things else: the foolishnesse of men may aske and muse why

was this so soone, and that so late? but the wisdom of God knowes what is fit for euery time: And surely amongst the sensible signes, and euident demonstrations of Gods all-gouerning providence, this is not the least, that he brings not forth his mightie works altogether, but *makes every thing beautifull in his time.* *Eccles. 3. 1.* And as in his creation he made not al at once, but produced them in their feuerall daies: so in his gubernation, he reueileth not the knowledge of all things in one Age, but discouers them in the feuerall ages of the World. And if man aske why God doth thus, holy *David* giues the answer; *The Lord hath so done his maruailous works, that they should be had in remembrance*; for were they all in one age (such is our corruption) they would bee lesse obserued and sooner forgotten, but being declared in their feuerall times, euery Age finds matter to magnifie God; And therefore He *whose glorious name is to be praised for ever*, reueils some meruailous thing in euery generation, that so his name may be praised from Generation to Generation. *Psal. 111. 4.* *Psal. 72. 19.*

TO THE TRVLY HO.

NORABLE AND RIGHT Wok
THY KNIGHT SIR Tiiom^s SMITHY
YR E A S F R E R for the Colonies and Conj
panics of V i & G 1 N 1 A: aod Gouernsur of Mtif.
couia, East-lildia North-weltPaffagc,
and S O M M E 9 Ilands
COM

N O R A B L 11 S 1 It, the W121 Of Men, Ot
rather the wifedome of God tells vs, thatibere
~r a time for a# tkings: and that the great Gor4,
who at his owne will beganne Time it fcl&,
dothathisowne timebeginne all things else:
the-foolithneffe ofinen may askc and tnufc why
was this fo roone, and that fo late? but the wifedome of God
knowes what is fit for euery time: And furely among(l the fen
fible signes, and cuidcnt demonfirations of Gods all-gouerning
providence, this is not the least, that he brings not fbrth his migh
tie works altogether, but makgrvffY thing boc#tifs.11 in Ur time.
rs.

And as in his creation he made not al at once, but produced them
in their feuerall daies: fo in his gubernation, hcreuedethnotthc
knowledge of all things in one Age, but difcouers them in the
fcuerall ages ofthe World. And if man aske why God doth thus,
holy *David* giues the answerc; *The Lord both fo done Ur mar-*
psal. 111. 4.

XML MARKUP

- XML (“Extensible Markup Language”) is a structured markup format to store text and metadata

```
<?xml version="1.0"?>
<forum type="viva">
<thread id="108738">
<category>Gezondheid</category>
<title>fibromyalgie?</title>
<posts>
<post id="0">
<author>marjaneke</author>
<timestamp>25-03-2011 15:16</timestamp>
<parentid></parentid>
<body>Hoi Allemaal,
```

Ik hoop dat er mensen zijn die mij kunnen helpen, al geruime tijd loop ik met vage klachten rond: pijnlijke schouders/nek met uitstraling naar armen, pijn achter mijn knieën en pijnlijke branderige voeten.

Ik ben met deze klachten naar de huisarts gegaan, hij heeft mij doorgestuurd naar de neuroloog, die stelde de diagnose fibromyalgie.

Sinds deze diagnose ben ik niet meer mezelf, ik speur heel internet af en wordt daar soms heel angstig van, vooral omdat ik lees dat fibromyalgie erg kan worden. Ook weet ik niet zeker of ik het echt heb, want ik slaap heel goed en ben verder niet moe.

Misschien wil ik niet accepteren dat ik deze ziekte heb, omdat het verloop zo kan verslechteren.

Wie heeft er tips, moet je perse bij fibro moe zijn en niet kunnen slapen of is dit per persoon anders? dank jullie wel voor de respons</body></post>

```
<post id="1">
<author>noanoanoa</author>
<timestamp>25-03-2011 15:20</timestamp>
<parentid></parentid>
<body>Tips om er mee om te gaan staan in dit boek:
```

[klik](<http://www.bol.com/nl/p/nederlandse-boeken/pijn-de-baas/666867803/index.html>)

```
Sterkte ![:rose:](<a href="http://forum.viva.nl/global/smileys/66_rose.gif">http://forum.viva.nl/global/smileys/66_rose.gif</a></body></post id="2">
<author>mariona</author>
<timestamp>25-03-2011 15:23</timestamp>
<parentid></parentid>
```

CLEAN TEXT STORAGE

- **Markup:** meta-information in a text file that is clearly distinguishable from the textual content
 - In the case of **XML and json**, markup often provides useful information in text processing
 - We typically convert PDF to XML, using pdf-to-xml convertors
 - Also, benchmark data is often stored as XML
- **Character Encoding:** the way that a computer displays text in a way that humans can understand

CHARACTER ENCODING

ASCII

- Character encoding: translates a string of 0s and 1s to a character
- ASCII is a 7-bit encoding based on the English alphabet

1100001	a
1100010	b
1100011	c
- ASCII: **American** Standard Code for Information Interchange

HOW ABOUT THE REST OF THE WORLD?

- corpus linguistics (English)
- corpuslinguïstiek (Dutch)
- कोष भाषा विज्ञान (Hindi)
- اللسانيات الإحصاء (Arabic)
- 語料庫語言學 (Chinese)
- בלשנות קורפוס (Hebrew)
- การ ศึกษา ภาษาศาสตร์ (Thai)
- Цорпус Лингвистиц (Serbian)

UNICODE

- Universal standard for all writing systems (>100,000 characters)
 - Independent of platform, software, vendor
- Interpretation of the character is done by the implementation (e.g. **UTF-8**) in the software (e.g. app, editor or web browser) that determines the actual rendering (size, shape, font, style)
- For maximum compatibility (forward and backward) we encode texts in UTF-8 when we read and write them

READ AND WRITE UTF-8 IN PYTHON 3

```
with open(filename, 'r', encoding='utf-8') as raw:  
    text = raw.read()
```

```
with open(filename, 'w', encoding='utf-8') as clean:  
    clean.write(text)
```

<https://docs.python.org/3/library/functions.html#open>

DATA CLEANING IN PRACTICE

- **Digitalization, data conversion & cleaning** are the first steps in the text mining process. These steps are:
 - necessary
 - time consuming
 - error prone
 - often complicated
- When building a text mining pipeline for new (raw) data, data collection and cleaning is a long and tedious step that is not to be underestimated!

REGULAR EXPRESSIONS

J&M SECTION 2.1

WHY REGULAR EXPRESSIONS

- We use regular expressions (regex) for finding patterns, e.g.
 - “[0-9][0-9][0-9][0-9] [A-Z][A-Z]”
 - “https?://\S+”
 - “\S+@\S+”
- This is common in preprocessing, e.g. for **removing noise, privacy sensitive information, or markup**
 - Match/count patterns in a file/string
 - Or extract the matched pattern

EXAMPLE REGEX

- Suppose we want to write a regular expression to find all occurrences of the English article *the*
- A simple (but incorrect) pattern might be: `/the/`
- **Why is this incorrect?**

J&M Section 2.1

EXAMPLE REGEX

- Suppose we want to write a regular expression to find all occurrences of the English article *the*
- A simple (but incorrect) pattern might be: `/the/`
- **Why is this incorrect?**
 - We want to find both *the* and *The*
 - But not: their, apothecary, etc.

J&M Section 2.1

EXAMPLE REGEX

- Instead, if we would really want to cover all occurrences, and no non-relevant occurrences of the string, we need:

```
/ (^ | [^a-zA-Z]) [tT]he ([^a-zA-Z] | $) /
```

- This soon becomes complex
- Alternative: use a Python package for proper text analysis
- **Spacy**

EXAMPLE IN SPACY

```
import spacy
import en_core_web_sm
spacy_nlp = en_core_web_sm.load()
spacy_doc = spacy_nlp(example_text)
spacy_words = [token.text for token in spacy_doc]
print(sum(token == 'the' for token in spacy_words))
```

TOKENIZATION AND SENTENCE SPLITTING

J&M SECTION 2.5 & 2.7

WHAT IS A WORD?

- How many words does the following sentence have?

This issue isn't just local—it's a global problem.

- This depends on your definition of word

This issue is not just local — it is a global problem .

- Now it is more clear:
 - 11 words and 2 punctuations
 - or: 13 **tokens**

J&M Section 2.2

DEFINITIONS

- **Token** – An instance of a word or punctuation occurring in a document
- **Word type** – a distinct word in the collection (without duplicates)
- **Term** – A token when used as feature (or in an index), generally in normalized form (e.g. lowercased)

- **Token count** is the number of words (running) in a collection/document; this includes duplicates
- **Vocabulary size $|V|$** is the number of unique terms (word types); the feature size when we use words as features

WHAT IS A WORD?

- How many **tokens** does the sentence have?
- And how many **word types**?

This issue is not just local — it is a global problem .

- 13 tokens and 10 word types
 - $|V| = 10$

J&M Section 2.2

TOKENIZATION

- Tokenization: Split text in tokens
- (1) remove punctuation; (2) split on whitespaces characters
- But: **how would you want to tokenize these strings?**
 1. State-of-the-art
 2. U.K.
 3. aren't
 4. C++
 5. \$1,000,000
 6. 20/03/2002

TOKENIZATION WITH SPACY

➤ <https://spacy.io/usage/linguistic-features#tokenization>

➤ Customization for your task possible

Editable Code

spaCy v3.7 · Python 3 · via Binder

```
import spacy

nlp = spacy.load("en_core_web_sm")
doc = nlp("This issue isn't just local-it's a global problem.")
for token in doc:
    print(token.text)
```

RUN

```
This
issue
is
n't
just
local
-
it
's
a
global
problem
.
```

SENTENCE SPLITTING

- We need sentence splitting for tasks that require **sentence-level analysis**, for example:
 - Extracting the relations between two entities (e.g. a person and their birth year) within one sentence
 - Sentence-level sentiment analysis (e.g. analyzing reviews for positive and negative aspects)
 - Input for Transformer models with limited input length

SENTENCE SPLITTING

- Many cases can be solved with a relatively simple approach:
 - Sentences end with . ? ! followed by whitespace
- **Challenges:**
 - Abbreviations ('Mrs. Doe')
 - Names/initials that include punctuation marks ('S. Verberne')
 - Sentences without punctuation markers (headers/titles)
 - Line endings inside sentences (PDF conversion)



SENTENCE SPLITTING WITH SPACY

➤ <https://spacy.io/usage/linguistic-features#sbd>

Editable Code

spaCy v3.0 · Python 3 · via Binder

```
import spacy

nlp = spacy.load("en_core_web_sm")
doc = nlp("This is a sentence. This is another sentence.")
assert doc.has_annotation("SENT_START")
for sent in doc.sents:
    print(sent.text)
```

RUN

```
This is a sentence.
This is another sentence.
```

SUBWORD TOKENIZERS

J&M SECTION 2.5

SUBWORD TOKENIZERS

“Instead of defining tokens as words (whether delimited by spaces or more complex algorithms), or as characters (as in Chinese) we can use our data to automatically tell us what the tokens should be”

- This is especially useful in **dealing with unknown words**
 - Remember from lecture 1 that if we have a large training collection, and add one document, the document can contain unseen words



SUBWORD TOKENIZERS

- Solution: automatically induce sets of tokens that include tokens smaller than words, called **subwords**
 - Byte-pair encoding (BPE, see J&M section 2.5.2)
 - WordPiece/SentencePiece
- Intuition: lower frequency words (that are not in the vocabulary) are **split up in tokens that are in the vocabulary**
- The subwords may not be words themselves
 - E.g. *This problem is proportionate* → *This problem is pro #portion #ate*

BYTE-PAIR ENCODING

Byte-pair encoding (BPE) token learner:

1. begin with a vocabulary that is the set of all individual characters
2. examine the training corpus, choosing the two symbols that are most frequently adjacent (e.g. 't', 'h'),
3. add a new merged symbol ('th') to the vocabulary
4. replace every adjacent 't', 'h' in the corpus with the new 'th'.
5. repeat step 2, creating new longer and longer character strings,
6. until k merges have been done creating k novel tokens (k is a parameter)

The resulting vocabulary consists of the original set of characters plus k new symbols

J&M Section 2.5.2

SUBWORD TOKENIZERS

WordPiece/SentencePiece: efficient implementations based on BPE

- Fixed vocabulary size (e.g. $k = 30k$ tokens)
- Optimize the selection of tokens to cover the whole language
 - Frequent words become their own token
 - (e.g. 'machine' -> 'machine')
 - Low-frequent words are split in subwords
 - (e.g. 'Gorlaeus' -> 'Go', '##rl', '##ae', '##us',)

https://huggingface.co/docs/transformers/main_classes/tokenizer

EXAMPLE

- **Technical text:** Our model works better with a large datastore at testing time and improves over the kNN-LM model (Khandelwal et al., 2020)
- **WordPiece:** Our model works better with a large data store at testing time and improves over the kNN-LM model (Khandelwal et al., 2020)

LEMMATIZATION AND STEMMING

(SECTION 2.6 IN J&M)

BASIC WORD FORMS

- We might want to normalize specific word forms to the same term:
 - For example, it can be useful to have the term *bicycle* for each occurrence of either *bicycle* or *bicycles*

BASIC WORD FORMS

- We might want to normalize specific word forms to the same term:
 - For example, it can be useful to have the term *bicycle* for each occurrence of either *bicycle* or *bicycles*
- Advantages:
 - reduces the number of features
 - generalizes better, especially for small datasets

BASIC WORD FORMS

- We might want to normalize specific word forms to the same term:
 - For example, it can be useful to have the term *bicycle* for each occurrence of either *bicycle* or *bicycles*
- Advantages:
 - reduces the number of features
 - generalizes better, especially for small datasets
- Two types of basic word forms:
 - **Lemma**
 - **Stem**

LEMMA

- Lemma: **dictionary form of a word**
- For example:
 - Verbs: infinitive
 - ‘think’ for ‘thinks’, ‘thinking’, ‘thought’
 - Nouns: singular form
 - ‘mouse’ for ‘mice’
 - ‘computer’ for ‘computers’

STEM

- Stem: the portion of a word that is common to a group of words **with the same prefix** (base/root)
 - Related words are mapped to the same stem
 - E.g. 'computer', 'computing', 'computers', 'compute'
- The stem is the part of the word that 'never' changes even when morphologically inflected. **It is not necessarily an existing word:**
 - 'comput' for forms 'computer', 'computing', 'computers', 'compute'

STEM VS LEMMA

- ‘produced’
 - what is the lemma?
 - what is the stem?

STEM VS LEMMA

- ‘produced’
 - what is the lemma?
 - what is the stem?
- Answers:
 - the lemma is ‘produce’
 - the stem is ‘produc’

(think of ‘producing’ as one of the morphological forms of the verb)
- **We almost always prefer lemmas over stems.** Stemming can be effective for very small collections

EXAMPLE

- **Sample text:** Such an analysis can reveal features that are not easily visible from the variations in the individual genes and can lead to a picture of expression that is more biologically transparent and accessible to interpretation
- **Lemmatizer:** Such an analysis can reveal feature that be not easily visible from the variation in the individual gene and can lead to a picture of expression that be more biologically transparent and accessible to interpretation
- **Stemmer:** such an analysi can reveal featur that ar not easili visibl from the variat in the individu gene and can lead to a pictur of express that is more biolog transpar and access to interpret

EDIT DISTANCE

J&M SECTION 2.8

WHY EDIT DISTANCE

For measuring string similarity:

- **Spelling correction/normalization** (think about normalizing the content of doctor's notes or user-generated content)
 - E.g. 'graffe' – what word was meant?
 - 'giraffe' differs by only one letter → most likely
 - 'grail' or 'graf' differ in more letters
- Also needed for normalizing and matching names to databases:
 - Gleevic → Gleevec (medication name)
 - Abbasiyantaeb → Abbasiantaeb (family name)

MINIMAL EDIT DISTANCE

- the **minimum edit distance** between two strings is defined as the minimum number of editing operations (insertion, deletion, substitution) needed to transform one string into another
- **Levenshtein distance**: insertion, deletion and substitution all have a cost of 1. Copying characters is free.
 - dog-do: 1
 - cat-cart: 1
 - cat-cut: 1
 - cat-act: ?
 - cat-act: 2

MINIMAL EDIT DISTANCE

Intuition of minimal edit distance:

- There are multiple ways to go from one word to another, but some require more edits than others
- E.g. to go from **sheep** to **ship**, we could:
 - copy 'sh', delete 'e', substitute 'e' by 'i', copy 'p' (edit distance = 2)
 - copy 'sh', delete 'e', delete 'e', insert 'i', copy 'p' (edit distance = 3)
 - the first option is 'cheaper' than the second

COMPUTING MINIMAL EDIT DISTANCE

- “The space of all possible edits is enormous, so we can’t search naively.
- However, lots of distinct edit paths will end up in the same state (string), so rather than recomputing all those paths, we could just remember the shortest path to a state each time we saw it.
- We can do this by using dynamic programming.
- **Dynamic programming** is the name for a class of algorithms that apply a table-driven method to solve problems by combining solutions to sub-problems.”

(J&M section 2.8.1)

LEVENSHTEIN DISTANCE: ALGORITHM

		f	a	s	t
c					
a					
t					
s					

- When we go to the right, we insert the next character of the horizontal word
- When we go down, we delete the next character of the vertical word
- When we go diagonal we substitute or copy
- We take the action with the minimum cost
- Until we end up at the right bottom corner

- Operations: insert (cost 1), delete (cost 1), substitute (cost 1), copy (cost 0)

LEVENSHTEIN DISTANCE: ALGORITHM

➤ Initialization

$$D(i, 0) = i$$

$$D(0, j) = j$$

➤ Recurrence Relation:

For each $i = 1 \dots M$

For each $j = 1 \dots N$

$$D(i, j) = \min \begin{cases} D(i-1, j) + 1 \\ D(i, j-1) + 1 \\ D(i-1, j-1) + \begin{cases} 1; & \text{if } X(i) \neq Y(j) \\ 0; & \text{if } X(i) = Y(j) \end{cases} \end{cases}$$

➤ Termination:

$D(N, M)$ is distance

X = string 1

Y = string 2

i = index in X

j = index in Y

$D(i, j)$ = value of cell (i, j)

LEVENSHTEIN DISTANCE: ALGORITHM

		f	a	s	t
c					
a					
t					
s					

- When we go to the right, we insert the next character of the horizontal word
- When we go down, we delete the next character of the vertical word
- When we go diagonal we substitute or copy
- We take the action with the minimum cost
- Until we end up at the right bottom corner

- Operations: insert (cost 1), delete (cost 1), substitute (cost 1), copy (cost 0)

LEVENSHTEIN DISTANCE: ALGORITHM

		f	a	s	t
	0	1	2	3	4
c	1	1	2	3	4
a	2	2	1	2	3
t	3	3	2	2	2
s	4	4	3	2	3

- The arrow follows the cheapest path, connecting the cheapest subpath to reach each next field

- Operations: insert (cost 1), delete (cost 1), substitute (cost 1), copy (cost 0)

LEVENSHTEIN DISTANCE: ALGORITHM

➤ Initialization

$$D(i, 0) = i$$

$$D(0, j) = j$$

➤ Recurrence Relation:

For each $i = 1 \dots M$

For each $j = 1 \dots N$

$$D(i, j) = \min \begin{cases} D(i-1, j) + 1 \\ D(i, j-1) + 1 \\ D(i-1, j-1) + \end{cases} \begin{cases} 1; & \text{if } X(i) \neq Y(j) \\ 0; & \text{if } X(i) = Y(j) \end{cases}$$

➤ Termination:

$D(N, M)$ is distance

X = string 1

Y = string 2

i = index in X

j = index in Y

$D(i, j)$ = value of cell (i, j)

LEVENSHTEIN DISTANCE: ALGORITHM

cost	operation	input	output
1	substitute	c	f
0	(copy)	a	a
1	substitute	t	s
1	substitute	s	t
Total: 3			

cost	operation	input	output
1	substitute	c	f
0	(copy)	a	a
1	deletion	t	*
0	(copy)	s	s
1	insertion	*	t
Total: 3			

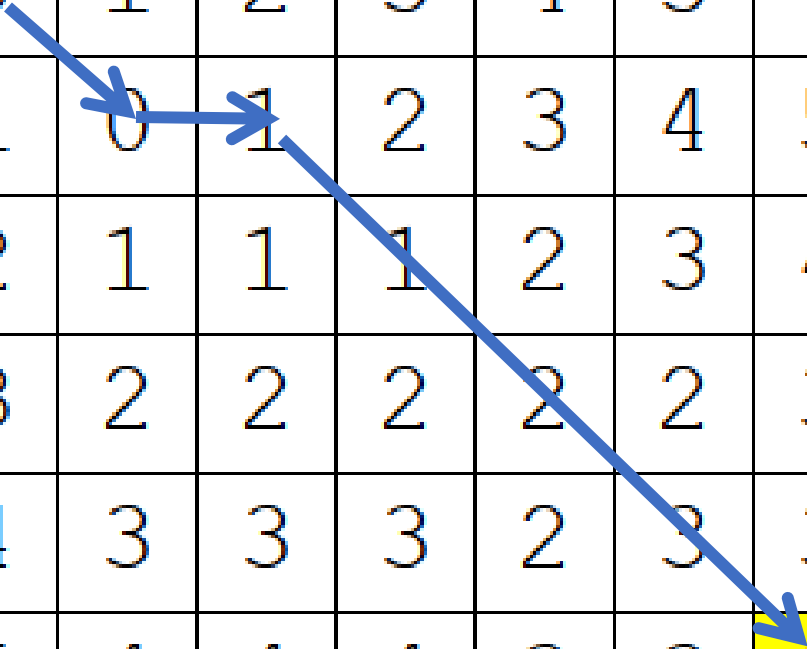


LEVENSHTEIN DISTANCE: EXERCISE

		a	n		a	c	t
	0	1	2	3	4	5	6
a	1						
	2						
c	3						
a	4						
t	5						

LEVENSHTEIN DISTANCE: EXERCISE

		a	n		a	c	t
	0	1	2	3	4	5	6
a	1	0	1	2	3	4	5
	2	1	1	1	2	3	4
c	3	2	2	2	2	2	3
a	4	3	3	3	2	3	3
t	5	4	4	4	3	3	3



LEVENSHTEIN DISTANCE: EXERCISE

cost	operation	input	output
0	(copy)	a	a
1	insert	*	n
0	(copy)		
1	substitute	c	a
1	substitute	a	c
0	(copy)	t	t

CONCLUSIONS

HOMework

- **Read Jurafsky & Martin** chapter 2: “Regular Expressions, Text Normalization, Edit Distance” (Brightspace)
- **Complete this week’s exercise:** Advanced NLP with Spacy, chapter 1 (Finding words, phrases, names and concepts, sections 1-12).
<https://course.spacy.io/en/chapter1>
- Contact address for the teaching assistants:
tmcourse@liacs.leidenuniv.nl

AFTER THIS LECTURE...

- you know what issues to take into account when converting raw text to **clean plain text**
- you can explain the difference between ASCII and **Unicode**; and why Unicode exists
- you can use **regular expressions** in Python and you know what the possibilities and challenges of regular expressions are
- you can describe the challenges of **tokenization and sentence splitting**
- you can explain the purpose of **subword tokenizers** and how they work
- you can define the difference between **stemming and lemmatization**
- you can compute the **minimal edit distance** between two strings using the matrix algorithm